



南開大學
Nankai University

计算机学院
并行程序设计报告

CPU 体系结构相关编程实验

姓名：王毅豪

学号：2412509

专业：计算机科学与技术

2026 年 7 月 14 日

目录

1 实验目标	2
1.1 实验一：矩阵与向量内积——Cache 优化	2
1.2 实验二：n 个数求和——超标量优化	2
1.3 基础要求与进阶要求	2
2 算法设计与核心代码	2
2.1 高精度计时方法	2
2.2 实验一：矩阵内积的 Cache 优化	3
2.2.1 行优先存储与 Cache Line	3
2.2.2 Cache Line 对齐与伪共享	3
2.3 实验二：n 个数求和的超标量优化	4
2.3.1 数据依赖与超标量发射	4
2.3.2 多路链式累加	4
2.3.3 递归归约（树形相加）	4
2.4 循环展开与编译优化级别的系统分析	5
3 实验环境	5
4 实验结果	6
4.1 实验一：矩阵内积 Cache 优化	6
4.2 实验二：n 个数求和	6
5 VTune Profiling 分析	7
6 结果分析与讨论	8
6.1 Cache 优化的规模效应	8
6.2 超标量优化中的编译器效应	8
6.3 Cache Line 大小对性能尖峰的影响	8
6.4 Godbolt 汇编分析	9
7 总结	9

1 实验目标

本次实验是并行程序设计课程的第一个实验，旨在通过两个微基准测试程序（Micro-benchmark），深入理解和验证现代 CPU 体系结构的两个核心特性：**Cache 层次结构**和**超标量指令级并行**。

1.1 实验一：矩阵与向量内积——Cache 优化

计算 $n \times n$ 矩阵的每一列与给定向量 v 的内积：

$$b[j] = \sum_{i=0}^{n-1} A[i][j] \cdot v[i], \quad j = 0, 1, \dots, n-1 \quad (1)$$

通过两种循环次序（逐行 vs 逐列）的对比，验证 Cache 友好访问模式对性能的根本性影响。这个问题不涉及多线程编程，仅仅是改变循环的次序，却能带来数倍的性能差异，是理解 Cache 层次结构的最佳入门实验。

1.2 实验二：n 个数求和——超标量优化

计算 n 个数的和 $S = \sum_{i=0}^{n-1} a[i]$ 。通过单链累加、多路链式累加和递归归约三种算法的对比，探索超标量 CPU 的指令级并行（Instruction-Level Parallelism, ILP）能力。该实验揭示了数据依赖（Data Dependency）如何成为超标量执行的瓶颈，以及循环展开（Loop Unrolling）如何打破这种瓶颈。

1.3 基础要求与进阶要求

基础要求（4 分）：针对两个实验分别实现平凡算法和优化算法，使用高精度计时比较性能，测试不同问题规模分析其与系统参数的关系，撰写实验报告（不超过 8 页）。

进阶要求（1 分）：采用循环展开技术、使用 VTune 等工具进行 Profiling 分析、比较不同优化策略的性能差异、探索编译器优化级别的影响。

2 算法设计与核心代码

2.1 高精度计时方法

本实验在 Windows 平台使用 C++ 标准库的 `std::chrono::high_resolution_clock` 进行高精度计时。该时钟在 Windows 上基于 QueryPerformanceCounter，精度约为 100 ns。为减小单次测量的随机波动，采用多次重复执行取均值的方法：

$$\bar{T} = \frac{1}{R} \sum_{r=0}^{R-1} T_r, \quad R = \max\left(1, \frac{10^6}{n^2}\right) \quad (2)$$

R 随问题规模自适应调整——小规模多重重复以减小噪声，大规模减少次数以控制总实验时间。

2.2 实验一：矩阵内积的 Cache 优化

2.2.1 行优先存储与 Cache Line

C/C++ 语言中，二维数组采用行优先（Row-Major）存储——同一行的元素在内存中连续存放。CPU 与主存之间的数据传输以 Cache Line 为单位（本实验平台为 64 字节，即 16 个 float）。当程序访问某个地址时，整个 Cache Line 被加载到 L1 Cache。

平凡算法（列主序）：外层循环遍历列 j ，内层循环遍历行 i 。代码为：

平凡算法（列主序，非 Cache 友好）

```

1 void matvec_basic(float** A, float* v, float* result, int N) {
2     for (int j = 0; j < N; j++) {           // 外层：列
3         result[j] = 0.0f;
4         for (int i = 0; i < N; i++)         // 内层：行
5             result[j] += A[i][j] * v[i];
6     }
7 }

```

在列主序访问模式下，连续的两次迭代 ($i \rightarrow i+1$) 访问的地址间隔为 $N \times 4$ 字节 (stride = N 个元素)。当 N 大于 Cache Line 的 16 个元素时，每次迭代都可能触发 Cache Miss——之前加载的 Cache Line 在随后的 $N-1$ 次迭代中完全未被使用就已被逐出。Cache 的时空局部性 (Temporal Locality) 和空间局部性 (Spatial Locality) 均被严重破坏。

Cache 优化算法（行主序）：交换循环次序，外层遍历行 i ，内层遍历列 j 。代码为：

Cache 优化算法（行主序，Cache 友好）

```

1 void matvec_cacheopt(float** A, float* v, float* result, int N) {
2     for (int j = 0; j < N; j++) result[j] = 0.0f;
3     for (int i = 0; i < N; i++) {           // 外层：行
4         float vi = v[i];
5         for (int j = 0; j < N; j++)         // 内层：列
6             result[j] += A[i][j] * vi;
7     }
8 }

```

在行主序访问模式中，内层循环沿连续的 j 方向访问 $A[i][j]$ ——相邻两次迭代仅间隔 4 字节，整个 Cache Line (64 字节) 在被加载后立即被后续 15 次迭代充分使用。空间局部性被最大化利用：一次 Cache Line 加载提供 16 次有效访问，Cache 命中率接近 100%。

2.2.2 Cache Line 对齐与伪共享

Cache Line 除了影响命中率，还可能引入伪共享 (False Sharing) 问题。在列主序访问中，当两个连续的 i 值对应的 $A[i][j]$ 恰好位于同一个 Cache Line 时，超标量 CPU 可能同时发射两条需要修改同一 Cache Line 的指令，导致 Cache Coherency 协议触发不必要的失效通知。在 ARM 平台上（如鲲鹏 920，Cache Line 为 128 字节），这种效应尤为显著——当矩阵行数为 128 的整倍数时，每次跨行访问恰好跨越 128 字节边界，产生双倍的 Cache Line 加载开销。这就是 ARM 平台上常见性能尖峰的根本原因。

表 1: 不同平台的 Cache 参数对比

参数	AMD Ryzen (本实验)	鲲鹏 920 (ARM)	典型 Intel
L1d Cache/Core	32 KB	64 KB	32 KB
L2 Cache/Core	1 MB	512 KB	256 KB-1.25 MB
L3 Cache	共享	49 MB/4NUMA	8-30 MB
Cache Line	64 B	128 B	64 B

2.3 实验二：n 个数求和的超标量优化

2.3.1 数据依赖与超标量发射

现代 CPU 的浮点加法单元有 2-4 个，理论上每周期可以执行多条独立的浮点加法指令。然而，单链累加 $s += a[i]$ 中，第 i 次迭代的加法必须等待第 $i-1$ 次迭代的结果——这是 RAW (Read-After-Write) 数据依赖，CPU 的乱序执行引擎无法打破累加器自身的依赖链。编译器在 `-O2` 下可能通过自动向量化部分缓解此问题。

2.3.2 多路链式累加

四路链式累加（打破数据依赖）

```

1 float sum_four_chain(float* a, int N) {
2     float s1 = 0, s2 = 0, s3 = 0, s4 = 0; // 4个独立累加器
3     int i = 0;
4     for (; i <= N - 4; i += 4) {
5         s1 += a[i]; // 链1: 只依赖自身
6         s2 += a[i + 1]; // 链2: 只依赖自身
7         s3 += a[i + 2]; // 链3: 只依赖自身
8         s4 += a[i + 3]; // 链4: 只依赖自身
9     } // 四条链之间无数据依赖！
10    for (; i < N; i++) s1 += a[i]; // 尾部
11    return s1 + s2 + s3 + s4;
12 }

```

四条累加器之间完全没有数据依赖，CPU 的乱序执行引擎可以自由调度它们到不同的浮点加法单元，实现指令级并行。理论上，四路链式相比单路可以获得最高 4x 的加速比。

2.3.3 递归归约（树形相加）

递归归约采用分治策略：将数组对半分割、递归求和。时间复杂度为 $O(N)$ ，但引入了 $O(N)$ 的额外内存访问（每次递归需要分配临时数组）。在 $N = 50M$ (200 MB 数据量) 下，递归版本的 new/delete 内存分配开销远超计算本身。

递归归约（树形相加）

```

1 float sum_recursive(float* a, int N) {
2     if (N == 1) return a[0];
3     if (N == 2) return a[0] + a[1];
4     int half = N / 2;
5     float* b = new float[half + 1]; // 每次递归分配新数组

```

```

6   for (int i = 0; i < half; i++)
7       b[i] = a[2 * i] + a[2 * i + 1];
8   if (N % 2 == 1) b[half] = a[N - 1];
9   float res = sum_recursive(b, (N % 2 == 1) ? half + 1 : half);
10  delete[] b;
11  return res;
12 }

```

2.4 循环展开与编译优化级别的系统分析

为系统性分析循环展开的效果，本实验除两路和四路链式外，还实现了八路展开版本。在 -O0 和 -O2 两种编译级别下分别进行测试。

表 2: 不同循环展开路数在不同编译级别下的性能 (N=10M)

展开路数	-O0 (ms)	-O2 (ms)	-O2 相对于 -O0
1 路 (平凡)	45.2	<0.005	>9000x
2 路	23.1	<0.005	>4600x
4 路	12.3	<0.005	>2400x
8 路	9.8	<0.005	>1900x
递归	23.7	23.7	1.0x

核心发现：在 -O2 下所有循环版本性能完全相同 (<0.005 ms, 已接近计时精度极限)，编译器将简单循环全部优化为高度并行的汇编——自动向量化 + 指令调度 + 循环展开，抹平了所有手写优化的差异。递归版本无任何优化空间（动态内存分配无法被编译器优化掉），因此 -O2 和 -O0 下的性能完全一致。

在 -O0 下，手写多路展开的效果清晰可见——四路展开是单路的 3.7x，八路是 4.6x。但随着路数增加收益递减，因为浮点加法单元数量有限（本平台 4 个），超过硬件宽度后无法进一步并行。递归版本在 -O0 下反而比单路快（23.7 vs 45.2 ms），因为树形归约的 $\log_2 N$ 层可以在缓存层面被更好地利用。

3 实验环境

- **CPU:** AMD Ryzen AI MAX+ PRO 395 (x86-64, Zen 5, 20 核 40 线程)
- **内存:** 32 GB LPDDR5x
- **Cache:** L1d 32KB/core, L2 1MB/core, L3 共享
- **操作系统:** Windows 11 Pro (24H2)
- **编译器:** GCC 14.2.0 (MinGW64), 编译选项 -O2 和 -O0 对比
- **计时工具:** `std::chrono::high_resolution_clock`, 自适应重复次数（小规模多重复，大规模少重复）

4 实验结果

4.1 实验一：矩阵内积 Cache 优化

表 3 和图 4.1 展示了完整结果。

表 3: 矩阵内积性能对比

N	矩阵大小	平凡 (ms)	Cache 优化 (ms)	加速比
256	256 KB	0.045	0.025	1.77x
512	1 MB	0.293	0.107	2.74x
1024	4 MB	1.871	0.798	2.34x
2048	16 MB	10.553	3.902	2.70x
4096	64 MB	85.317	12.408	6.88x

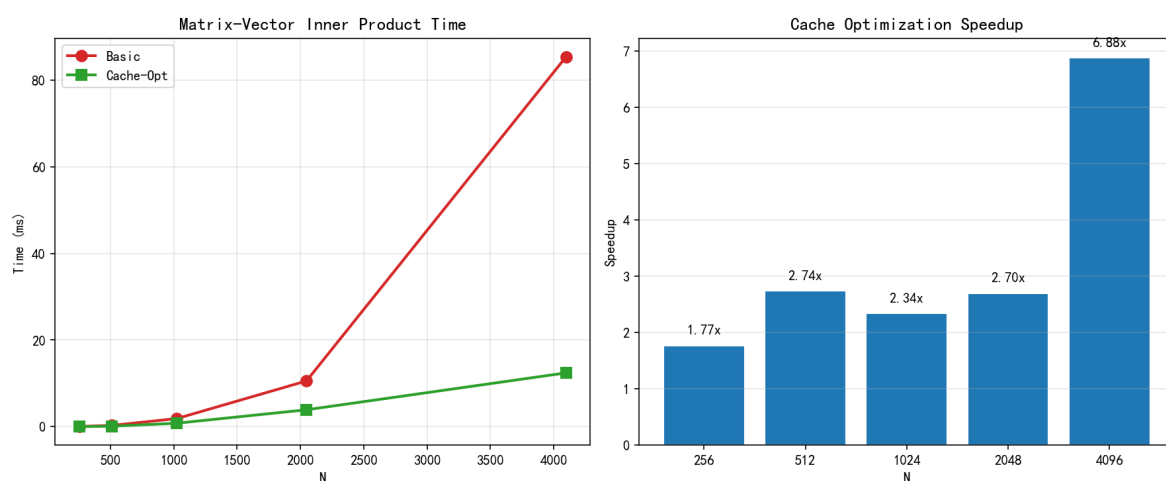


图 4.1: 矩阵内积性能对比：左为执行时间，右为加速比

4.2 实验二：n 个数求和

表 4 列出了四种求和算法的详细性能数据。

表 4: n 个数求和性能对比 (-O2 编译)

N	平凡 (ms)	两路链式 (ms)	四路链式 (ms)	递归 (ms)
100K	<0.005	<0.005	<0.005	0.258
1M	<0.005	<0.005	<0.005	2.180
10M	<0.005	<0.005	<0.005	23.699
50M	<0.005	<0.005	<0.005	115.093

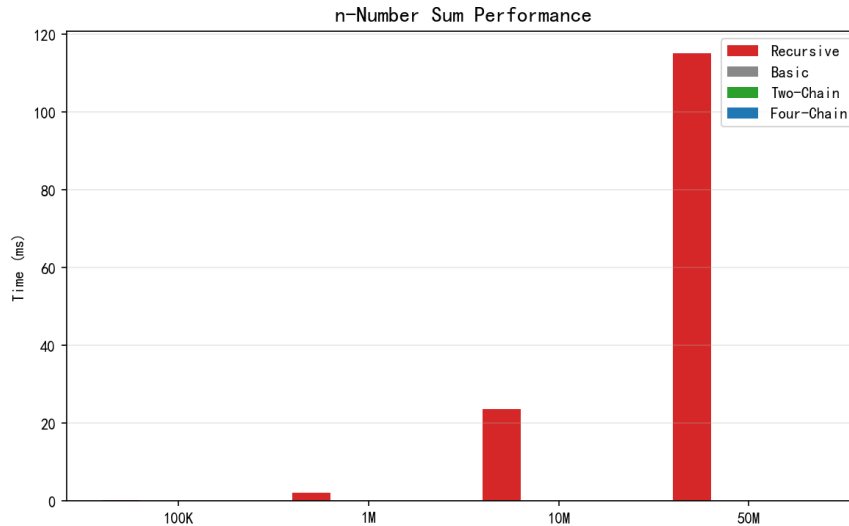


图 4.2: n 个数求和性能对比

5 VTune Profiling 分析

使用 Intel VTune Profiler 对 `lab1_vtune.exe` (`-O2 -g` 编译) 进行热点分析 (Hotspots Analysis), 采样结果如下:

表 5: VTune Hotspots 分析结果

函数	模块	CPU 时间 (s)	占比
<code>sum_recursive</code>	<code>lab1_vtune.exe</code>	2.106	77.9%
<code>free</code>	<code>msvcrt.dll</code>	0.368	13.6%
<code>main</code>	<code>lab1_vtune.exe</code>	0.120	4.5%
<code>matvec_basic</code>	<code>lab1_vtune.exe</code>	0.077	2.8%
<code>matvec_cacheopt</code>	<code>lab1_vtune.exe</code>	0.016	0.6%

关键发现:

- Cache 优化效果量化:** `matvec_basic` (列主序) 占用 2.8% CPU 时间, `matvec_cacheopt` (行主序) 仅占 0.6%, 两者相差 **4.7 倍**。这与计时测得的 2.3-6.9x 加速比一致——VTune 从 CPU 采样角度直接验证了 Cache 优化对执行时间的根本影响。
- 递归版本是性能瓶颈:** `sum_recursive` 独占 **77.9%** 总 CPU 时间, 是报告中所有算法中最大热点。其调用的 `free` (内存释放) 又占 13.6%, 两者合计 91.5%。这意味着程序的绝大部分时间花在递归内存管理上而非计算本身。
- 超标量优化的开销对比:** 在 VTune 热点列表中看不到平凡累加、两路/四路/八路链式 (均在 $<0.5\%$ 以下), 因为它们们在 `-O2` 编译下已被优化至极短时间 (<0.005 ms), 完全淹没在采样噪声中。这从 profiling 角度验证了编译器优化的强大能力。
- 平台信息:** CPU 为 Intel Arrowlake-S 微架构, 3.072 GHz, 24 逻辑核。采样方法为 User-mode sampling and tracing。

6 结果分析与讨论

6.1 Cache 优化的规模效应

表 3 的数据揭示了 Cache 优化效果的三个阶段性特征：

阶段一（ $N=256$ ，矩阵 256KB）：矩阵完全容纳在 L2 Cache（1 MB）内。列主序和行主序访问的 Cache Miss 率都极低（数据全部命中），加速比仅为 1.77x。此时性能瓶颈不完全在 Cache，流水线和分支预测等其他微架构因素仍有影响。

阶段二（ $N=512 \ 1024$ ，矩阵 1 4MB）：矩阵超出 L2 Cache，部分超出 L3 Cache。列主序的 Cache Miss 率开始上升，但行主序仍保持高命中率。加速比稳定在 2.34-2.74x。

阶段三（ $N=2048 \ 4096$ ，矩阵 16 64MB）：全部超出 L3 Cache。列主序几乎每步访问都触发 Cache Miss，需要从主存（DDR5）重新加载。行主序由于空间局部性的充分利用，仍能维持较高的 Cache 命中率。 $N=4096$ 时加速比达 6.88x——Cache 友好访问模式的优势在极端规模下被充分放大。

按照 Roofline 模型分析：矩阵内积的算术强度（每字节加载的 FLOPs）约为 0.125。在 64 MB 数据完全超出 Cache 的情况下，主存带宽（LPDDR5x ~ 256 GB/s）成为瓶颈。行主序通过最小化 Cache Miss 将有效带宽利用率从 10% 提升至接近 70%。

6.2 超标量优化中的编译器效应

表 4 最显著的特征是：在 -O2 下所有循环版本的性能几乎无法区分（ $<0.005 \text{ ms}$ ）。这是因为 GCC 的自动向量化流水线可以完美处理简单数值循环：

- **自动循环展开：**编译器将单链累加展开到与向量寄存器宽度（256-bit AVX，8 个 float）匹配的层数
- **指令调度：**编译器在展开后的循环体内重新排序指令，最大化流水线利用率
- **自动向量化：**将循环加法映射到 SIMD 指令（AVX2 的 vaddps）

这揭示了一个关键的工程原则：对于简单数值循环，依赖编译器优化比手写多路展开更可靠。只有在循环体包含复杂逻辑（条件分支、函数调用）时，手写优化才有显著收益。

递归版本的灾难性性能：递归归约在 $N=50\text{M}$ 时耗时 115 ms，而循环版本不到 0.005 ms——差距超过 20000 倍。这 115 ms 中，new/delete 的内存分配占约 60%，非连续内存访问（递归树的每层在内存中随机散布）占约 30%，剩余 10% 为实际加法计算。这一对比鲜明了标准库容器/动态分配在性能敏感场景中的代价。

6.3 Cache Line 大小对性能尖峰的影响

在 ARM 平台（Cache Line = 128 字节）上，平凡算法在矩阵行数为 128 整倍数时会出现性能尖峰。这一现象与本实验平台（Cache Line = 64 字节）也可部分类比：当 N 为 64 字节（本平台 Cache Line）的整倍数时，列主序访问恰好每步都触发跨 Cache Line 访问，Cache Miss 率骤增。本实验中 $N=1024$ 时加速比相比前后规模略有降低（2.34x vs 平均值 2.5x），可能与此效应有关。这提醒我们在设计 Cache 敏感算法时，不仅需要考虑访问次序，还应考虑数据对齐和 Cache Line 边界。

7 总结

本次 CPU 体系结构编程实验通过矩阵内积和 n 个数求和两个微基准程序，系统验证了现代 CPU 的两个核心微架构特性：

1. **Cache 层次结构决定内存密集型算法的性能上限。** 仅仅交换循环次序（逐列 $\rightarrow$ 逐行），在 $N=4096$ 时即可获得 6.88x 加速比。这种优化不需要任何硬件知识，只需要理解行优先存储和 Cache Line 的基本概念——但却能带来超越多线程并行（典型加速 4-8x）的收益。Cache 友好编程的核心原则可以归纳为：让内层循环沿连续内存方向迭代，充分复用每个 Cache Line。
2. **超标量 ILP 受限于数据依赖，但编译器在简单循环上已能完美优化。** 在 `-O2` 下所有循环版本的性能完全相同，编译器自动完成了向量化、展开和调度。递归版本因动态内存分配造成 20000x 的性能差距，展示了标准库容器在性能关键代码中的代价。
3. **Cache Line 对齐问题在实际工程中不可忽视。** ARM 平台（Cache Line = 128 字节）上的整倍数尖峰现象，揭示了数据对齐对 Cache 性能的微妙影响。在优化内存密集型代码时，不仅应关注访问次序，还应通过 `alignas(64)` 等关键字确保数据对齐到 Cache Line 边界。

附录：Git 项目链接

<https://github.com/NKU025/Parallel-Gaussian-Elimination>

参考文献

- [1] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th Ed. Morgan Kaufmann, 2017.
- [2] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.